

Detección de Outliers

Clasificación y Métodos de Clustering

Méndez Ávila Luis Geovanni Paredes Zamudio Luis Daniel
Sánchez Rosas Roberto Samuel

2026-05-29

Tabla de contenidos

1	¿Qué es un Outlier?	3
1.1	¿Por qué es importante detectar los outliers?	3
1.2	Outliers y Ruido	3
2	Categorización de Outliers	4
2.1	Outliers Globales	4
2.2	Outliers Contextuales	4
2.3	Outliers Colectivos	4
3	¿Qué significa <i>clustering</i>?	5
3.1	¿De qué nos sirven los clústers?	5
4	Algoritmos de Clustering	6
4.1	Términos Generales de los Algoritmos de Clustering	6
4.1.1	Ejemplo Corto	6
4.2	K-Means	6
4.2.1	Visualizando K-Means	7
4.2.2	La Regla del Codo	7
4.2.3	Ejemplo de Uso: El consumo eléctrico en hogares	10
5	Referencias	13

1 ¿Qué es un Outlier?

Es un objeto que tiene características diferentes de la mayoría de otros del dataset con el que estemos trabajando. También puede ser un valor inusual con respecto a los valores típicos de una variable.

Por ejemplo, si tenemos un dataset con la información de la altura de un grupo de personas, y el promedio es de 1.80 metros, un outlier sería ver un valor de 2.15 metros.

La naturaleza de los outliers es dual, así como pueden ser el resultado de errores de medición o entrada de datos, también pueden representar eventos genuinos pero poco frecuentes que aún nos pueden dar información valiosa del conjunto de datos. Munoz (2025)

1.1. ¿Por qué es importante detectar los outliers?

Si estos valores no se identifican ni procesan de una manera adecuada perjudica considerablemente los análisis estadísticos que se hagan de nuestros datos, ya que puede sesgar el modelo, reducir su precisión y/o comprometer su interpretabilidad.

Sin embargo la decisión de cómo manejar un outlier debe precederse por una investigación sobre *por qué existe*. Si un outlier es claramente el resultado de un error su eliminación o corrección es justificada. Sin embargo, si el outlier representa un evento genuino y válido, pero tal vez menos probable, debe ser considerado cuidadosamente o incluso aprovechado para obtener información valiosa. «Clustering» (2026)

1.2. Outliers y Ruido

Algo que vale la pena recalcar es que un outlier **no es lo mismo** que el ruido. Mientras que el ruido son datos incorrectos o inconsistentes que restan la calidad del modelo que se vaya a crear, un outlier es más un valor que es posible pero muy poco frecuente.

2 Categorización de Outliers

2.1. Outliers Globales

Son los outliers menos específicos. Son eventos que se desvían significativamente del resto del dataset. Por ejemplo, un conjunto de datos de edad de personas en una empresa y un dato anómalo de 150 años.

2.2. Outliers Contextuales

Estos eventos se consideran atípicos según el contexto específico bajo el que se vayan a tratar. Por ejemplo, si nuestro dataset es sobre temperaturas en un estado a lo largo del año, un valor de 30°C es normal en verano, pero atípico en invierno.

2.3. Outliers Colectivos

Como su nombre lo dice, son eventos que aunque de manera individual no parecen ser anómalos, en conjunto si representan una anomalía. Por ejemplo, el rechazo de envío de un paquete de una computadora a otra se considera normal, pero si varias computadoras continúan enviando paquetes rechazados unas a otras, el conjunto se debería considerar como outlier colectivo. Gaona et al. (2021)

3 ¿Qué significa *clustering*?

El clustering es una técnica de aprendizaje automático no supervisado que busca dividir el dataset en el que se este trabajando en grupos, a los que llamaremos clústers, en donde los elementos que se encuentran dentro de uno son lo más similares entre si y lo más distintos posibles o los de otro clúster.

Como es un método no supervizado no se necesita que los datos tengan etiquetas, clasificaciones previas o ejemplos de entrenamiento, si no que más bien lo que se busca es encontrar la distancia matemática (o similitud) entre cada dato para agruparlos lo más correctamente posible.

3.1. ¿De qué nos sirven los clústers?

- **Descubrir patrones ocultos:** Ayuda a revelar relaciones, estructuras y agrupaciones naturales dentro de los datos que no son evidentes a simple vista.
- **Simplificar información compleja:** Permite dividir y resumir grandes volúmenes de datos ruidosos en subgrupos más pequeños, homogéneos y manejables.
- **Exploración y descubrimiento:** Es sumamente útil para encontrar grupos de datos o categorías que eran previamente desconocidas por los analistas.

4 Algoritmos de Clustering

4.1. Términos Generales de los Algoritmos de Clústering

1. El algoritmo evalúa cada registro del conjunto de datos tomando en cuenta sus características como si fueran coordenadas de un punto en el espacio.
2. Luego, usamos métricas (como la distancia euclídea o la correlación) para medir qué tan cerca están esos puntos unos de otros
3. Con base en esta proximidad, el algoritmo asocia los puntos de modo que se minimice la distancia entre los elementos de un mismo grupo (haciéndolos muy compactos y similares) y se maximice la separación con los demás grupos.

4.1.1. Ejemplo Corto

Digamos que tenemos un dataset de compras de un supermercado. Cada entrada en el conjunto se refiere a lo que compra una persona y queremos detectar grupos basados en clustering en base a esos datos. Entonces el algoritmo puede agrupar en “compradores frecuentes”, “compradores nuevos”, “compradores de un solo artículo”, etc. Y en base a estas métricas el supermercado puede encontrar patrones de los artículos más vendidos, etc.

4.2. K-Means

El algoritmo de K-Means es uno de los métodos de clustering más populares y ampliamente utilizados. Su objetivo es dividir un conjunto de datos en k clústers distintos, donde k es un número predefinido por el usuario. Funciona de la siguiente manera:

- Tomamos k datos (o puntos) aleatorios como los centros iniciales (o *centroides*) de cada clúster.
- Asignamos el resto de los datos a el clúster más cercano calculando su distancia a cada centro y asignándolo al de menor distancia.
- Una vez que todos los datos han sido asignados a un clúster, el algoritmo recalcula los centros de cada clúster tomando la media de los puntos asignados a ese clúster.
- Repetimos los pasos de asignación y recalcado de centros hasta que las asignaciones de los puntos no cambien significativamente o se alcance un número máximo de iteraciones.

4.2.1. Visualizando K-Means

Por ejemplo, en las siguientes imágenes tenemos un dataset con una n cantidad de datos. Vamos a tomar $k = 3$ y seleccionamos 3 puntos aleatorios y los marcamos con azul, verde y amarillo. De esto creamos los primeros clústeres basándonos en la distancia de cada punto al centroide. Luego, recalculamos los centroides tomando la media de los puntos asignados a cada clúster y repetimos el proceso hasta que los clústeres no cambien significativamente. «¿Qué es el agrupamiento en clústeres k-means?» (2025)

4.2.2. La Regla del Codo

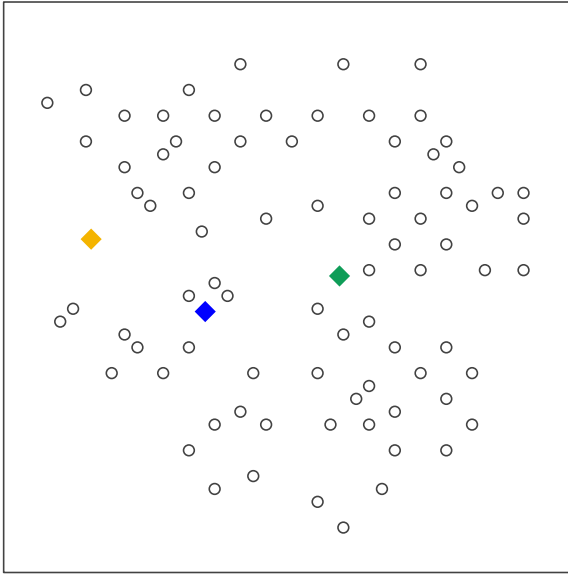
¿Y cómo sabemos cual es el valor correcto de k a considerar? El método del codo sirve justo para esto. Funciona de la siguiente manera:

- Ejecutamos k -Means para diferentes valores de k (por ejemplo, desde 1 hasta 10).
- Para cada valor de k , calculamos la *inercia*, que es la suma de las distancias al cuadrado entre cada punto y su centroide asignado. Esta inercia representa qué tan bien se ajustan los datos a los clústers formados.
- Luego, graficamos la inercia en función de k y buscamos el punto donde la disminución de la inercia comienza a ser menos pronunciada, formando una especie de codo (de ahí el nombre).
- El punto resultante suele ser la mejor elección para el número óptimo de clústers, ya que representa un buen equilibrio entre la complejidad del modelo (número de clústers) y la calidad del ajuste (inercia).

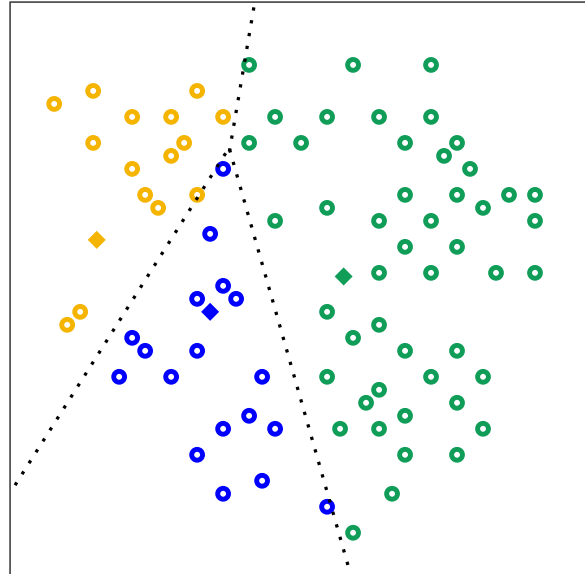
4.2.2.1. Ejemplo usando el método del codo

Vamos a crear un conjunto de datos aleatorios usando la función `make_blobs` de `sklearn.datasets`, que genera datos agrupados en clústers. Luego, aplicaremos el método del codo para determinar el número óptimo de clústers. Consideraremos que el número máximo de clústers a evaluar es 10, pero esto puede ajustarse según las necesidades del análisis.

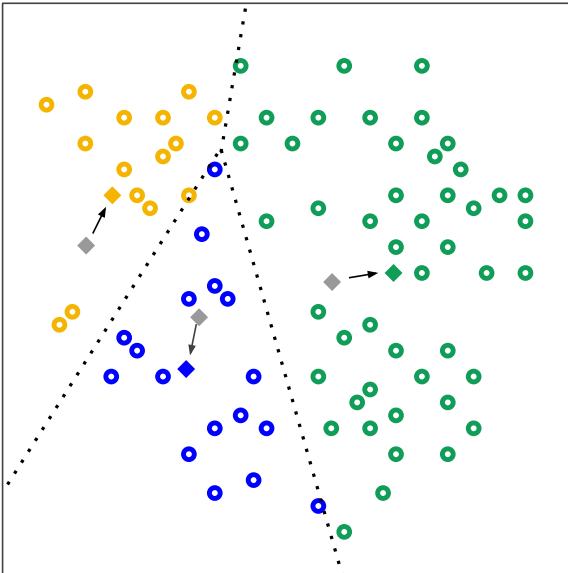
```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.cluster import KMeans
5 from sklearn.datasets import make_blobs
6 import plotly.express as px
7 from sklearn.preprocessing import StandardScaler
8
```



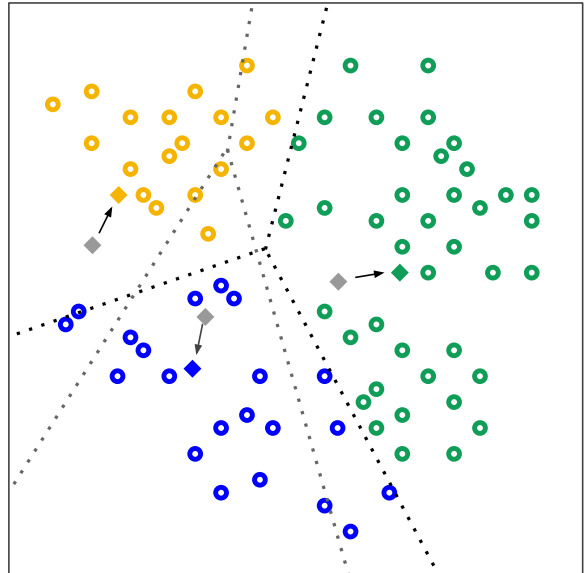
(a) Inicialización de K-Means con $k = 3$



(a) Clústeres Iniciales



(a) Centroides recalculados

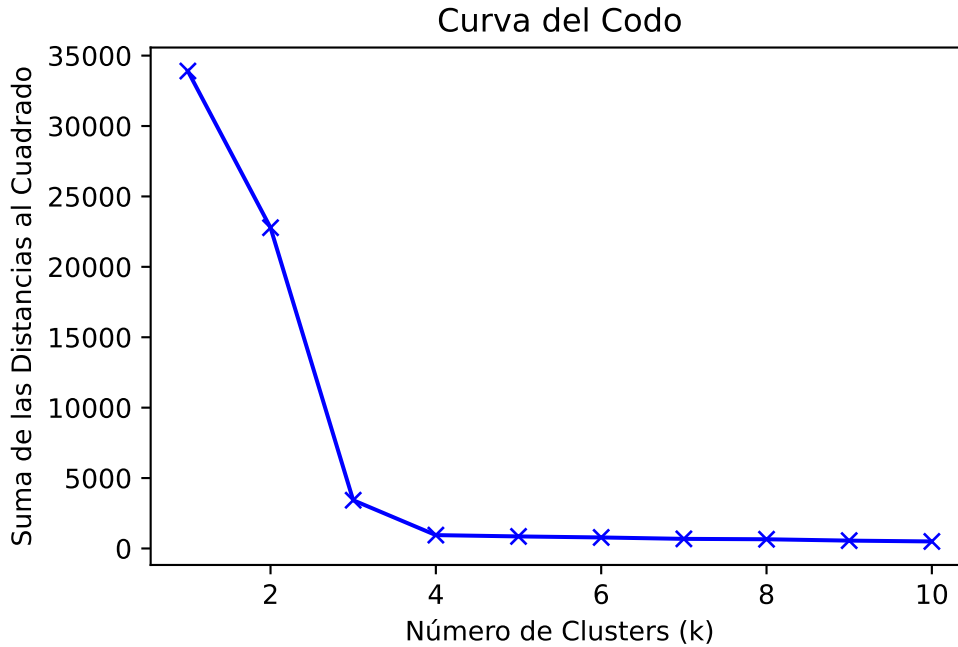


(a) Clústeres Finales


```

9  def elbow_method(data, max_clusters=10):
10     """
11     Aplica el método del codo para seleccionar el número óptimo de clusters en k-means.
12
13     Parámetros:
14     -----
15     data : matriz o matriz dispersa, forma (n_samples, n_features)
16           Los datos de entrada.
17
18     max_clusters : int, opcional (por defecto=10)
19                   El número máximo de clusters a considerar.
20     """
21
22     sum_of_squared_distances = []
23
24     for k in range(1, max_clusters+1):
25         kmeans = KMeans(n_clusters=k)
26         kmeans.fit(data)
27         sum_of_squared_distances.append(kmeans.inertia_)
28
29     # Graficar la curva del codo
30     plt.plot(range(1, max_clusters+1), sum_of_squared_distances, 'bx-')
31     plt.xlabel('Número de Clusters (k)')
32     plt.ylabel('Suma de las Distancias al Cuadrado')
33     plt.title('Curva del Codo')
34     plt.show
35
36     # Genera un conjunto de datos de ejemplo con 4 clusters
37     data, _ = make_blobs(n_samples=500, centers=4, n_features=2, random_state=42)
38     # Aplica el método del codo
39     elbow_method(data)

```



En este ejemplo, vemos que el valor se reduce significativamente hasta $k = 4$, lo que sugiere que 4 es el número óptimo de clústers para este conjunto de datos. Rodríguez (2025)

4.2.3. Ejemplo de Uso: El consumo eléctrico en hogares

Para ilustrar el funcionamiento de k-means en la detección de datos atípicos, analizaremos un dataset real de consumo eléctrico doméstico, proveniente del [repositorio UCI](#). Este dataset contiene mediciones minuto a minuto de la potencia activa consumida y el voltaje registrado en una vivienda francesa desde Diciembre de 2006 hasta Noviembre de 2010.

Al aplicar k-means a este dataset, podemos identificar patrones de consumo y detectar posibles outliers que podrían indicar anomalías en el consumo eléctrico, como picos inusuales o caídas repentinas.

1. Selección y limpieza de variables:

Se utilizan dos variables principales:

- `Global_active_power` (potencia activa global, en kW)
- `Voltage` (voltaje de la red, en V)

Se eliminan valores nulos y se convierten los datos a formato numérico.

2. Escalamiento:

Dado que ambas variables tienen escalas muy distintas, se normalizan para evitar que una

domine el análisis de distancias. Usaremos `StandardScaler` de `sklearn.preprocessing` para centrar los datos en torno a la media y escalar a la unidad de varianza.

3. Aplicación de k-means:

Se agrupan los datos en k clústeres (el número de clústeres es ajustable). Para cada registro, se calcula la distancia al centroide de su grupo. Usaremos el algoritmo de k-means de `sklearn.cluster` para realizar esta tarea.

4. Identificación de atípicos:

Los registros cuya distancia al centroide supera un percentil alto (por ejemplo, el 99 %) se marcan como atípicos.

- **Atípicos globales:** Potencias inusualmente altas, sin importar el voltaje.
- **Atípicos contextuales:** Consumos normales pero asociados a voltajes anómalos.

5. Visualización:

Se genera un gráfico interactivo donde los puntos atípicos se resaltan en rojo, permitiendo explorar visualmente la estructura y los extremos del dataset.

Los resultados mostrados a continuación son fijos debido a la naturaleza de Quarto y Jupyter, pero se puede consultar [este cuaderno de Marimo](#) para ver un análisis interactivo al modificar la cantidad de clústers y el umbral de detección de atípicos.

```
1 df = pd.read_csv('datasets/household_power_consumption.txt', sep=';', nrows= 10000, low_me
2 df_limpio = df[['Global_active_power', 'Voltage']].replace('?', np.nan).dropna().astype(fl
3 scaler = StandardScaler()
4 df_scaled = scaler.fit_transform(df_limpio)
5 kmeans = KMeans(n_clusters=4, random_state=42)
6 df_limpio['Cluster'] = kmeans.fit_predict(df_scaled)
7 # Obtenemos las coordenadas de los centroides y calculamos la distancia de cada punto a su
8 centroides = kmeans.cluster_centers_
9 distancias = np.sqrt(np.sum((df_scaled - centroides[df_limpio['Cluster']]) ** 2, axis=1))
10 # El porcentaje de los datos más lejanos a cualquier clúster se consideran atípicos
11 umbral = np.percentile(distancias, 99) # Ajustable según el nivel de sensibilidad deseado
12 df_limpio['Es_Atipico'] = distancias > umbral
13
14 # Creamos una columna de texto para formatear las leyendas de forma clara
15 df_grafico = df_limpio.copy()
16 df_grafico['Clasificación'] = df_grafico['Cluster'].astype(str)
17 df_grafico.loc[df_grafico['Es_Atipico'], 'Clasificación'] = 'Atípico'
18
19 # Mapa de colores: clústeres comunes y rojo para anomalías
20 mapa_colores = {
21     '0': '#1f77b4',      # Azul
```

```

22     '1': '#2ca02c',      # Verde
23     '2': '#9467bd',      # Morado
24     '3': '#bcbd22',      # Amarillo
25     '4': '#17becf',      # Cian
26     '5': '#e377c2',      # Rosa
27     '6': '#ff7f0e',      # Naranja
28     '7': '#8c564b',      # Café
29     '8': '#7f7f7f',      # Gris
30     '9': '#ffbb78',      # Naranja claro
31     'Atípico': '#d62728' # Rojo para Outliers
32 }
33
34 # Construcción del scatter plot interactivo
35 fig = px.scatter(
36     df_grafico,
37     x='Global_active_power',
38     y='Voltage',
39     color='Clasificación',
40     color_discrete_map=mapa_colores,
41     symbol='Clasificación',
42     symbol_sequence=['circle', 'x'], # Los normales serán círculos y los atípicos serán 'x'
43     opacity=0.6,
44     title='Detección de Atípicos (K-Medias)',
45     labels={
46         'Global_active_power': 'Potencia Activa Global (kW)',
47         'Voltage': 'Voltaje (V)'
48     },
49     hover_data={'Cluster': True, 'Es_Atipico': False} # Muestra datos limpios al pasar el
50 )
51
52 fig.update_layout(
53     legend_title_text='Estructura del Dataset',
54     margin=dict(l=40, r=40, t=60, b=40)
55 )

```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

5 Referencias

- Anblicks. 2020. «Types of outliers: Key to accurate data analysis». En *Anblicks*. <https://www.anblicks.com/blog/an-introduction-to-outliers-what-are-outliers-types-of-outliers/>.
- «Clustering: Segmentación de Datos para Descubrir Patrones Ocultos». 2026. En *Pontia Tech*. <https://www.pontia.tech/clustering-que-es/>.
- Gaona, Amparo, Gerardo Áviles, y Salvador López. 2021. «Detección de anomalías». En *Minería de datos con R*, 1.^a ed. Las Prensas de Ciencias.
- IBM. 2025. «¿Qué es el clustering?» En *IBM*. <https://www.ibm.com/es-es/think/topics/clustering>.
- Munoz, Harold. 2025. «Datos Atípicos (outliers) en machine learning: Impacto, detección y aplicaciones en detección de fraude». En *Blog de Tecnología, Datos, Machine Learning y Big-Data*. <https://disenet.com/datos-atipicos-outliers-en-machine-learning-impacto-deteccion-y-aplicaciones-en-deteccion-de-fraude/>.
- Pei, Jian. 2026. *Data Mining - Outlier Detection*. <https://www2.cs.sfu.ca/CourseCentral/459/jpei/21/459OutlierDetection.pdf>.
- «¿Qué es el agrupamiento en clústeres k-means?» 2025. En *Google*. Google. https://developers.google.com/machine-learning/clustering/kmeans/overview?hl=es-419#k-means_clustering_algorithm.
- Rodríguez, Daniel. 2025. «Método del Codo (elbow method) para seleccionar el número óptimo de clústeres en k-means». En *Analytics Lane*. <https://www.analyticslane.com/2023/06/09/metodo-del-codo-elbow-method-para-seleccionar-el-numero-optimo-de-clusteres-en-k-means/>.